

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been copied and communicated to you by or on behalf of The University of Newcastle pursuant to Part VB of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

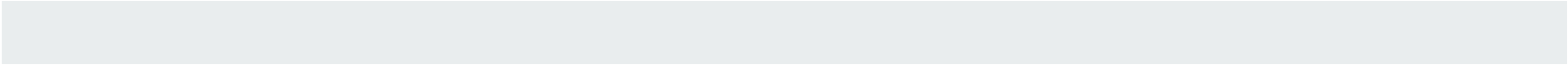
SENG1050

Web Technologies

Week 10 - Encoding and Compression

Lecture Outline

- ☐ Encoding
- ☐ Error Checking
- ☐ Data Compression
- ☐ Statistical Compression
- ☐ Substitutional Compression
- ☐ Compressing Multimedia



Recall from last week: Website Design with HCI and SEO

- ❑ Last week, we looked at how to make our website attractive to human and computer audiences, using HCI principles for human audiences, and SEO principles for computer audiences.
- ❑ We looked at designing for accessibility, and how to layout a website in a manner that is easy to navigate.
- ❑ We saw how to test the load speed of our websites, how to use the Lighthouse analysis tool, and how to optimise our images, CSS, and JavaScript to speed up our page load times.
- ❑ This revealed another problem – larger websites are still loading too slowly, even with these optimisations. Ultimately the best way to load a website faster is to transfer a smaller amount of data.
- ❑ We want to shrink the size of our files so that they can be sent faster. Today, we will look at how this is done, and the basics of how this data is represented on a computer system.

Encoding

Encoding

- ❏ In general: The conversion of information into another form
- ❏ For computers: The conversion of information into binary data
 - 1s and 0s = bits
- ❏ Encoding is not new:
 - The Telegraph and Morse Code...

Morse Code

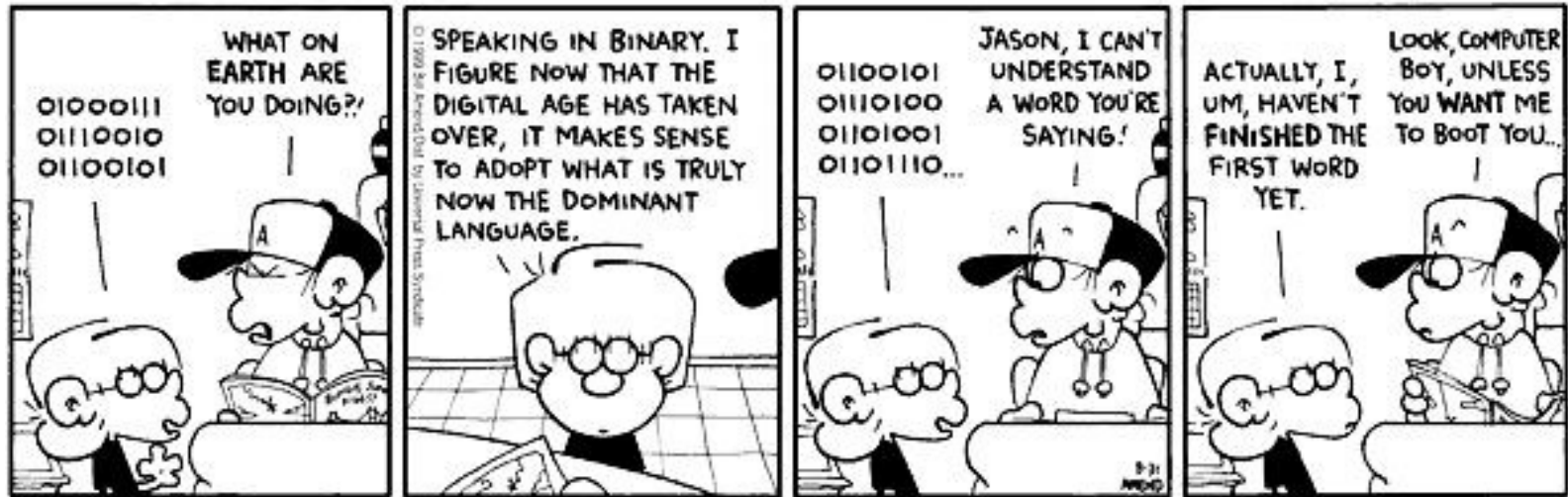
- ❑ 1836, Samuel Morse
- ❑ Alphabet:
 - Letters ⇒ dots (short click, *dit*) and dash (long clicks, *dah*)
 - Spaces (between words) ⇒ pauses
- ❑ Customers don't need to know the rules!
 - Only the operator

http://en.wikipedia.org/wiki/Morse_code

Morse Code

A	.-	G	--.	M	--	S	...	Y	-.--	4	-
B	-...	H		N	-.	T	-	Z	--..	5	
C	-.-.	I	..	O	---	U	..-	0	-----	6	-.....
D	-..	J	.---	P	..-.	V	...-	1	.-----	7	--.....
E	.	K	-. -	Q	--.-	W	.- -	2	..---	8	---..
F	...-	L	.-...	R	.-.	X	-...-	3	...--	9	----.

- ❑ Based on the distribution of letters in daily usage
- ❑ Short codes are assigned to more frequent letters
- ❑ Overall, fewer dots-dashes are needed to encode (and thus transfer) information



What is Jason saying?

ASCII

- ❑ Alphabet = {0, 1}
- ❑ Most common encoding(?) = **American Standard Code for Information Interchange**
 - Encodes English characters – uppercase and lowercase letters, numbers, punctuation, space and some “special characters” (tab, return, linefeed, etc.)
 - Uses 7 bits / character $\Rightarrow$ 128 values
- ❑ Examples:
 - 'A' = 1000001, 'B' = 1000010, 'C' = 1000011,
 - 'a' = 1100001, 'b' = 1100010, '!' = 0100001

ASCII Table

ASCII Hex Symbol	ASCII Hex Symbol	ASCII Hex Symbol	ASCII Hex Symbol
0 0 NUL	16 10 DLE	32 20 (space)	48 30 0
1 1 SOH	17 11 DC1	33 21 !	49 31 1
2 2 STX	18 12 DC2	34 22 "	50 32 2
3 3 ETX	19 13 DC3	35 23 #	51 33 3
4 4 EOT	20 14 DC4	36 24 \$	52 34 4
5 5 ENQ	21 15 NAK	37 25 %	53 35 5
6 6 ACK	22 16 SYN	38 26 &	54 36 6
7 7 BEL	23 17 ETB	39 27 '	55 37 7
8 8 BS	24 18 CAN	40 28 (	56 38 8
9 9 TAB	25 19 EM	41 29)	57 39 9
10 A LF	26 1A SUB	42 2A *	58 3A :
11 B VT	27 1B ESC	43 2B +	59 3B ;
12 C FF	28 1C FS	44 2C ,	60 3C <
13 D CR	29 1D GS	45 2D -	61 3D =
14 E SO	30 1E RS	46 2E .	62 3E >
15 F SI	31 1F US	47 2F /	63 3F ?

ASCII Hex Symbol	ASCII Hex Symbol	ASCII Hex Symbol	ASCII Hex Symbol
64 40 @	80 50 P	96 60 ^	112 70 p
65 41 A	81 51 Q	97 61 a	113 71 q
66 42 B	82 52 R	98 62 b	114 72 r
67 43 C	83 53 S	99 63 c	115 73 s
68 44 D	84 54 T	100 64 d	116 74 t
69 45 E	85 55 U	101 65 e	117 75 u
70 46 F	86 56 V	102 66 f	118 76 v
71 47 G	87 57 W	103 67 g	119 77 w
72 48 H	88 58 X	104 68 h	120 78 x
73 49 I	89 59 Y	105 69 i	121 79 y
74 4A J	90 5A Z	106 6A j	122 7A z
75 4B K	91 5B [	107 6B k	123 7B {
76 4C L	92 5C \	108 6C l	124 7C
77 4D M	93 5D]	109 6D m	125 7D }
78 4E N	94 5E ^	110 6E n	126 7E ~
79 4F O	95 5F _	111 6F o	127 7F 

Unicode

- ❑ ASCII only covers “English characters”
- ❑ Unicode supports over 1 million “characters” covering nearly all known languages
 - UTF-32 – uses 32 bits per character
 - UTF-16 – uses 16 bits per character
 - UTF-8 – uses 8 bits per character (compatible with ASCII)
 - <http://www.unicode.org/versions/Unicode4.1.0/>

Error Checking

Error Checking

- ❏ Things go wrong:
 - Faults in wiring
 - Interference from other wires
 - Interference from other devices
 - Interference from natural phenomena (lightning, sun spots)
- ❏ ...can change the value of one or more bits



Parity

- ❑ Byte = bits needed to store a character = 8
 - But ASCII only uses 7 bits!
- ❑ 8th (parity) bit can be used for error checking
 - Doesn't *only* apply to ASCII
- ❑ Add all the 1s in the 7 bit string:
 - Even-parity: sum is odd $\Rightarrow$ 8th bit = 1
 sum is even $\Rightarrow$ 8th bit = 0
 - Odd-parity: sum is odd $\Rightarrow$ 8th bit = 0
 sum is even $\Rightarrow$ 8th bit = 1
- ❑ Even parity is mostly used.
- ❑ Used for keyboards, serial lines, etc.

Parity

- Example:

1000011 = 'C'	even-parity $\Rightarrow$ 11000011 <i>«interference»</i>
1000010 = 'B'	11000010 $\Rightarrow$ parity error

- Example:

1000011 = 'C'	even-parity $\Rightarrow$ 11000011 <i>«interference»</i>
0000000 = NUL	00000000 $\Rightarrow$ parity okay

- The Internet uses more powerful error checking...



Cyclic Redundancy Check (CRC)

- ❑ Method of detecting errors in the communication channel.
- ❑ Uses **Generator Polynomial, $G(x)$** , which is available on both the sender and receiver.
 - $G(x)$ is a polynomial expression such as $x^3 + x + 1$, $x^2 + 1$
 - These keys/generators can be represented in binary form
 - $x^3 + x + 1$ is 1011, $x^2 + 1$ is 101
 - $x^3 + x + 1 \Rightarrow 1x^3 + 0x^2 + 1x + 1 \Rightarrow 1011$
- ❑ Performs **modulo-2 binary division** to divide the binary data by the binary key. The remainder of this division is used for checking errors.
- ❑ CRC with $G(x) = 11$ is equivalent to odd-bit parity check



Cyclic Redundancy Check (CRC)

We have a generator with length k (number of bits)

At the sender:

1. Append $k-1$ zeros to the binary data.
2. Perform modulo-2 binary division to divide the binary data with the generator key and get the remainder. The remainder is always $k-1$ bits long.
3. Add the remainder to the original data, and transmit the data.

At the receiver:

1. Perform modulo-2 binary division on the data with the generator key again.
2. If the remainder is zero, there's no error. Otherwise, there's an error.

Cyclic Redundancy Checking (CRC)

Frame/Message: 11 01 01 10 11

Generator Polynomial: $x^4 + x + 1 \Rightarrow 1x^4 + 0x^3 + 0x^2 + 1x + 1 \Rightarrow 10\ 01\ 1$

Generator/Key: 10 01 1 (5 bits long)

Message after appending $5-1 = 4$ zeros: **11 01 01 10 11 0000**

Divide the message by the Key (10 01 1) to get the remainder

10 01 1 | 11 01 01 10 11 0000

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

$$\begin{array}{r} \underline{1} \\ 10\ 01\ 1 \mid 11\ 01\ 01\ 10\ 11\ 0000 \\ 10\ 01\ 1 \end{array}$$

```
Frame/Message: 11 01 01 10 11
Generator/Key : 10 01 1
```

Generator/Key : 10 01 1

$$\begin{array}{r}
 1 \\
 \hline
 10 \ 01 \ 1 \mid 11 \ 01 \ 01 \ 10 \ 11 \ 0000 \\
 10 \ 01 \ 1 \downarrow \\
 \hline
 1 \ 00 \ 11
 \end{array}$$

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

	<u>11</u>	
10 01 1	11 01 01 10 11 0000	
	10 01 1	
	1 00 11	
	1 00 11	

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

```
      11
10 01 1 | 11 01 01 10 11 0000
      10 01 1
      1 00 11
      1 00 11
      1 00 11
      00 00 1
```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

```
      110
10 01 1 | 11 01 01 10 11 0000
      10 01 1
      -----
        1 00 11
        1 00 11
        -----
          00 00 1
          00 00 0
```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

Diagram illustrating the long division of 110 by 1011. The quotient is 10110 and the remainder is 10. The steps show the divisor 1011 being subtracted from the dividend 11000000 in a series of steps, with the final remainder 10 highlighted in red.

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

```
      1100
10 01 1 | 11 01 01 10 11 0000
      10 01 1
      1 00 11
      1 00 11
      00 00 1
      00 00 0
      0 00 10
      0 00 00
```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

```

      1100
10 01 1 | 11 01 01 10 11 0000
      10 01 1
      1 00 11
      1 00 11
      00 00 1
      00 00 0
      0 00 10
      0 00 00
      00 10 1

```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

```

      11000
10 01 1 | 11 01 01 10 11 0000
      10 01 1
      -----
        1 00 11
        1 00 11
        -----
          00 00 1
          00 00 0
          -----
            0 00 10
            0 00 00
            -----
              00 10 1
              00 00 0
```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

```

      11000
10 01 1 | 11 01 01 10 11 0000
      10 01 1
      1 00 11
      1 00 11
      00 00 1
      00 00 0
      0 00 10
      0 00 00
      00 10 1
      00 00 0
      0 10 11

```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

```

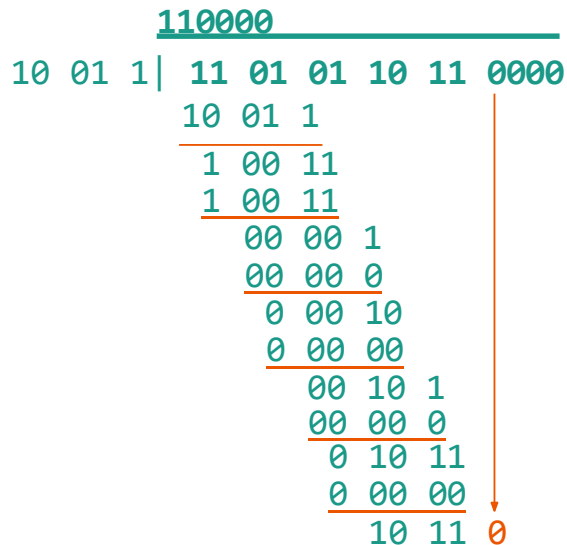
      110000
10 01 1 | 11 01 01 10 11 0000
      10 01 1
      1 00 11
      1 00 11
      00 00 1
      00 00 0
      0 00 10
      0 00 00
      00 10 1
      00 00 0
      0 10 11
      0 00 00
```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**



The diagram illustrates the long division process for CRC calculation. The divisor is 10 01 1, and the dividend is 11 01 01 10 11 0000. The process involves a series of XOR operations (represented by subtraction) to shift the divisor across the dividend. The final remainder is 0.

```

      110000
      -----
10 01 1 | 11 01 01 10 11 0000
          10 01 1
          -----
            1 00 11
            1 00 11
            -----
              00 00 1
              00 00 0
              -----
                0 00 10
                0 00 00
                -----
                  00 10 1
                  00 00 0
                  -----
                    0 10 11
                    0 00 00
                    -----
                      10 11 0

```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

```

      1100001
10 01 1 | 11 01 01 10 11 0000
      10 01 1
      -----
        1 00 11
        1 00 11
        -----
          00 00 1
          00 00 0
          -----
            0 00 10
            0 00 00
            -----
              00 10 1
              00 00 0
              -----
                0 10 11
                0 00 00
                -----
                  10 11 0
                  10 01 1
```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

```

      1100001
10 01 1 | 11 01 01 10 11 0000
      10 01 1
      -----
        1 00 11
        1 00 11
        -----
          00 00 1
          00 00 0
          -----
            0 00 10
            0 00 00
            -----
              00 10 1
              00 00 0
              -----
                0 10 11
                0 00 00
                -----
                  10 11 0
                  10 01 1
                  -----
                    0 10 10
```

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

1100001

10 01 1 | 11 01 01 10 11 0000

10 01 1

1 00 11

1 00 11

00 00 1

00 00 0

0 00 10

0 00 00

00 10 1

00 00 0

0 10 11

0 00 00

10 11 0

10 01 1

0 10 10

Continue this process until you get the final remainder **1110**

Error Checking - Cyclic Redundancy Checking

Frame/Message: 11 01 01 10 11

Generator/Key : 10 01 1

Message after appending 0 CRC: **11 01 01 10 11 0000**

1100001

10 01 1 | 11 01 01 10 11 0000

10 01 1

1 00 11

1 00 11

00 00 1

00 00 0

0 00 10

0 00 00

00 10 1

00 00 0

0 10 11

0 00 00

10 11 0

10 01 1

0 10 10

Continue this process until you get the final remainder **1110**

Transmitted message:
11 01 01 10 11 1110

Data Compression

Data Compression

- ❑ "Storing data in a format that requires less space than usual".
- ❑ Used to increase the amount of data that can fit on a storage device.
 - Less important these days.
- ❑ Used to decrease the number of bits needed to send information over a network connection.
 - Still very important.
 - Trades transmission time (expensive) for CPU time (cheap).

Morse Code

- ❑ Compression is not new: Morse code...
- ❑ Based on the distribution of letters in daily usage.
- ❑ **Short codes are assigned to more frequent letters.**
- ❑ Overall, fewer dots-dashes are needed to encode (and thus transfer) information.

A	.-	G	--.	M	--	S	...	Y	-.--	4	-
B	-...	H		N	-.	T	-	Z	--..	5	
C	-.-.	I	..	O	---	U	..-	0	-----	6	-....
D	-..	J	.----	P	.-.-	V	...-	1	.----	7	--...
E	.	K	-.-	Q	--.-	W	.-.-	2	.-----	8	---..
F	..-.	L	.-..	R	.-.	X	-..-	3	...--	9	----.

Compression and Archive

- ❑ A **compressor** compresses a **single file**
 - E.g., compress, gzip, bzip2, zip
- ❑ An **archiver** combines **several input files** into a **single output file**
 - Many archivers compress the individual files before combining
 - E.g., arc, arj, lha, zip, zoo, rar, jar, war
 - Some archivers do not
 - E.g., tar, ar
 - A compressor is often applied to the result – e.g., tgz
- ❑ Most archivers/compressors add **error checking**



Compression Techniques

Lossless Compression Techniques

- ❑ The integrity of the data is preserved. The original data and the data after compression and decompression are exactly **the same**.
- ❑ Because, in these methods, the compression and decompression algorithms are exact inverses of each other: no part of the data is lost in the process.
- ❑ Redundant data is removed in compression and added during decompression.
- ❑ Lossless compression methods are normally used when we cannot afford to lose any data.



Compression Techniques

Lossy Compression Techniques

- ❑ Our eyes and ears cannot distinguish subtle changes.
- ❑ In such cases, we can use a **lossy** data compression method.
- ❑ These methods are cheaper
 - ❑ they take less time and space when it comes to sending millions of bits per second for images and video.



Compression Techniques

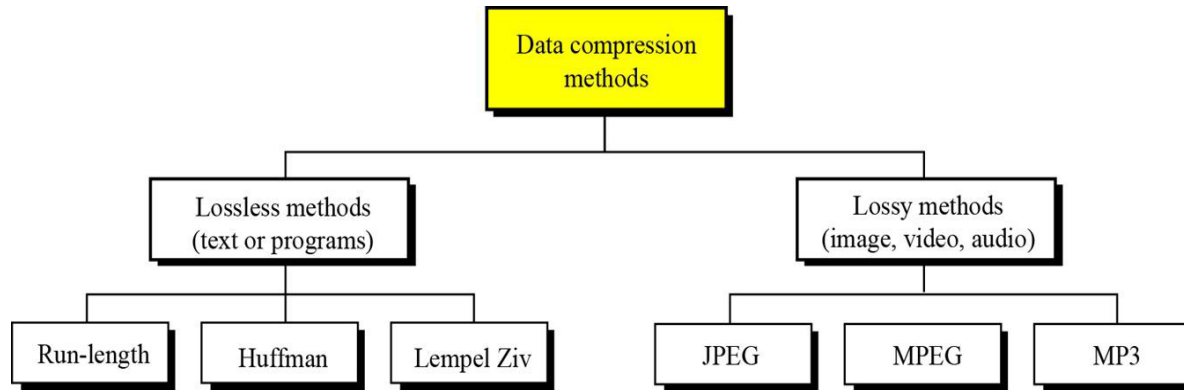
Lossless Compression Techniques

- ❑ Data (D) → Compression → Compressed Data (C) → Decompression → Data (D)
 - **$\text{uncompress}(\text{compress}(\text{data})) = \text{data}$**
- ❑ Statistical data
- ❑ Text documents
- ❑ Executable file

Lossy Compression Techniques

- ❑ Data (D) → Compression → Compressed Data (C) → Decompression → Decompressed Data (E)
 - **$\text{uncompress}(\text{compress}(\text{data})) \approx \text{data}$**
- ❑ Audio
- ❑ Video

Compression Techniques



Statistical Compression

Run-Length Encoding Compression

- ❑ Assumption: consecutive elements (bit, character, etc.) are likely to be the same
- ❑ Useful for encoding **cartoons**, or black and white images like faxes
- ❑ Repeating characters are replaced with a shorter representation

```
aaaaaa  
aabcdde  
eeeeeee  
efggghi  
iijjkk  
kkkkkkk
```

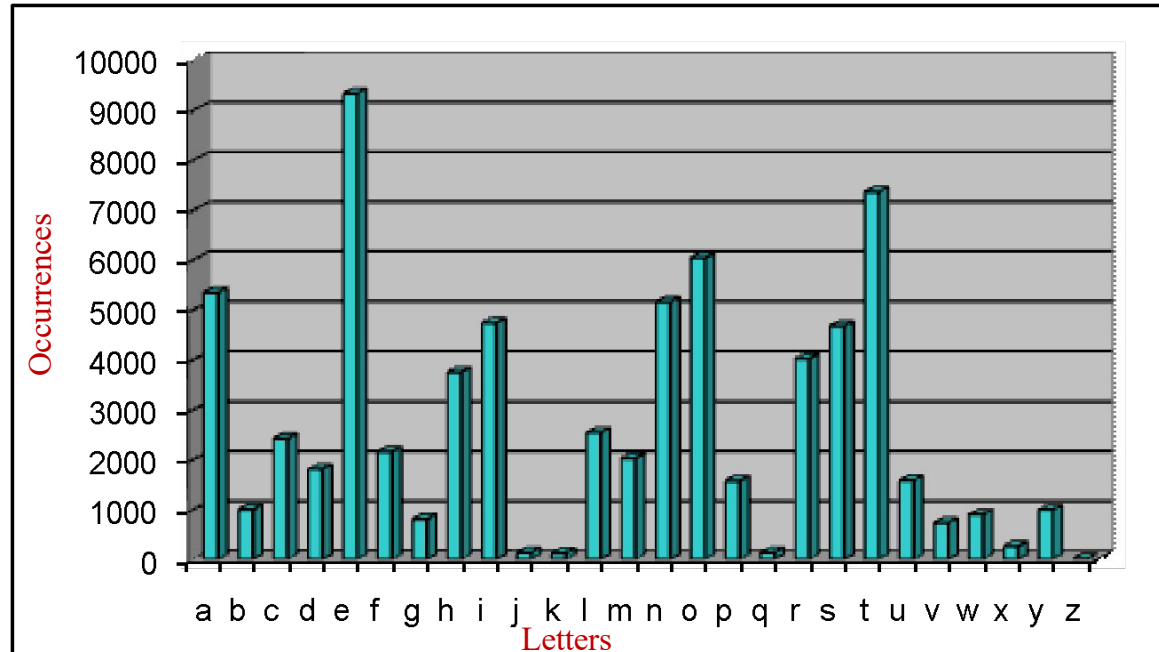
- Explicitly encode values

9a1b1c1d10e1f3g1h3i3j9k

22/42 bytes

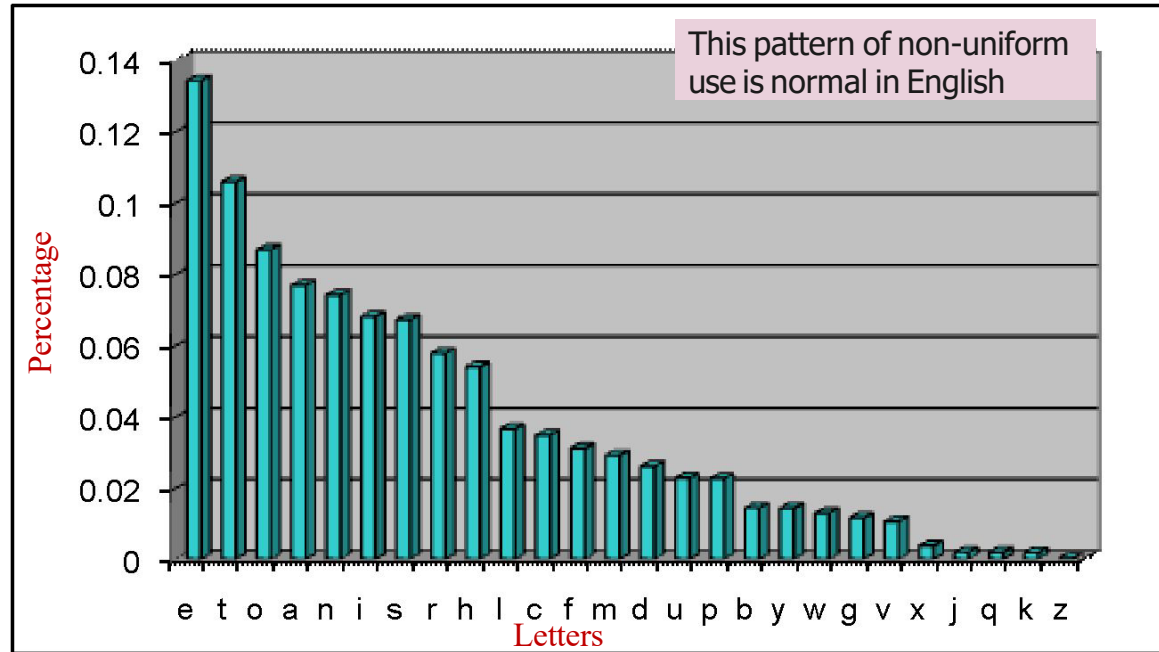
Statistical Compression

Aaron Quigley 1/5/2000: case ignored



Statistical Compression

Aaron Quigley 1/5/2000: case ignored





Huffman Encoding

- ❑ A statistical, variable-length data compression technique
- ❑ Reduces the **average** code length used to represent the symbols of an alphabet
- ❑ Variable-length encoding can have ambiguities when decoding
 - ❑ a=0, b=010, c=01, d=10
 - ❑ If the message is 00101001
 - ❑ Is it?
 - ❑ 0 0 10 10 01 => aaddc
 - ❑ 0 010 10 01 => abdc
- ❑ We need codes for each character, such that code for a character is not a prefix of another code.
 - ❑ Such codes are called *prefix codes*.
 - ❑ In the above example, a=0 is a prefix of b=010. So this will create confusion
- ❑ Before we compress the data with Huffman encoding, we need to determine the code (called the Huffman code)



Huffman Encoding

To construct a Huffman code:

1. Perform an analysis that first rank all symbols by frequency of occurrence.
2. Repeatedly combine the two symbols of the **lowest probability** to form a new composite symbol – this builds **a binary tree** where each node is labelled with the frequency of all nodes beneath it.
3. Follow the “trail” to each leaf – left is **0** right is **1**.
4. This trail for each letter is it's Huffman Coding.

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d	1808	0.079062
b	997	0.043598
g	803	0.035115
Total	22868	100%

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d	1808	0.079062
b	997	0.043598
g	803	0.035115
Total	22868	100%

} First
Combine
forms bg

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d	1808	0.079062
(bg)	1800	0.078712
Total	22868	100%

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d	1808	0.079062
(bg)	1800	0.078712
Total	22868	100%

Next Combine
forms d and
(bg)

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d(bg)	3608	0.157775
Total	22868	100%

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d(bg)	3608	0.157775
Total	22868	100%

Next
Combine
forms n and f

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
nf	4577	0.200148
d(bg)	3608	0.157775
Total	22868	100%

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
nf	4577	0.200148
d(bg)	3608	0.157775
Total	22868	100%

Next Combine
forms nf and
d(bg)

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
((nf)(d(bg)))	8185	0.357923
Total	22868	100%

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
((nf)(d(bg)))	8185	0.357923
Total	22868	100%

} Next Combine
forms a and
((nf)(d(bg)))

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a((nf)(d(bg)))	13530	0.591656
Total	22868	100%

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a((nf)(d(bg)))	13530	0.591656
Total	22868	100%

Finally Combine
forms e and
a((nf)(d(bg)))

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d	1808	0.079062
b	997	0.043598
g	803	0.035115
Total	22868	100%

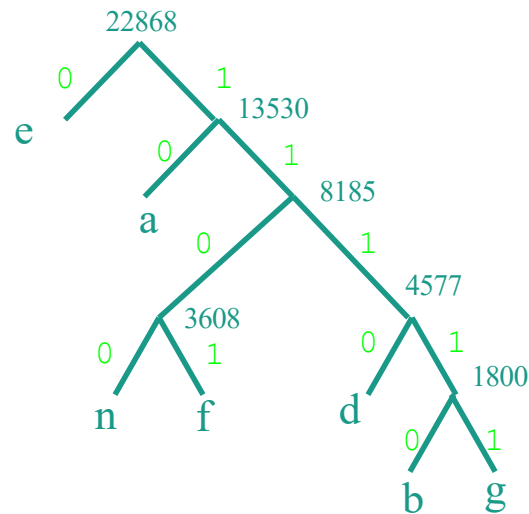
e(a((nf)(d(bg))))



Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d	1808	0.079062
b	997	0.043598
g	803	0.035115
Total	22868	100%

e(a((nf)(d(bg))))

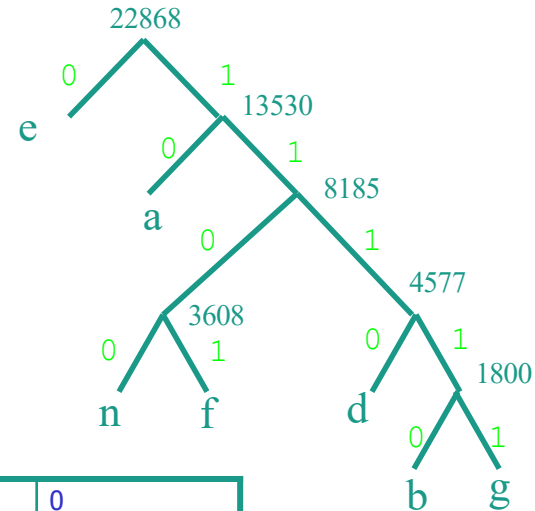


Create the binary tree
from the bottom up, i.e.,
starting with 'b' and 'g'
(top down is alright too!)

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d	1808	0.079062
b	997	0.043598
g	803	0.035115
Total	22868	100%

e(a((nf)(d(bg))))

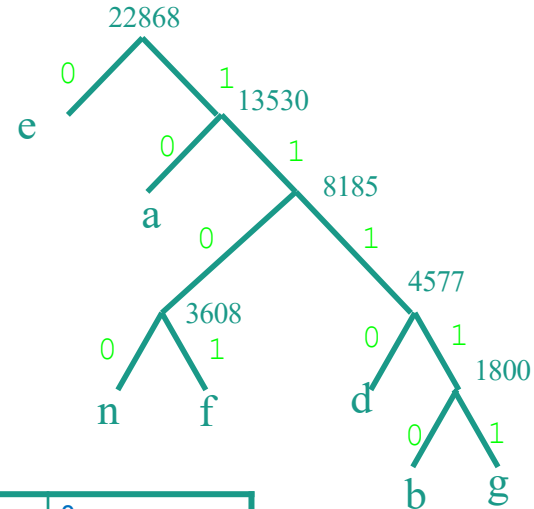


e	0
a	10
n	1100
f	1101
d	1110
b	11110
g	11111

Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d	1808	0.079062
b	997	0.043598
g	803	0.035115
Total	22868	100%

Try decoding this:
1111010110010110010



e	0
a	10
n	1100
f	1101
d	1110
b	11110
g	11111

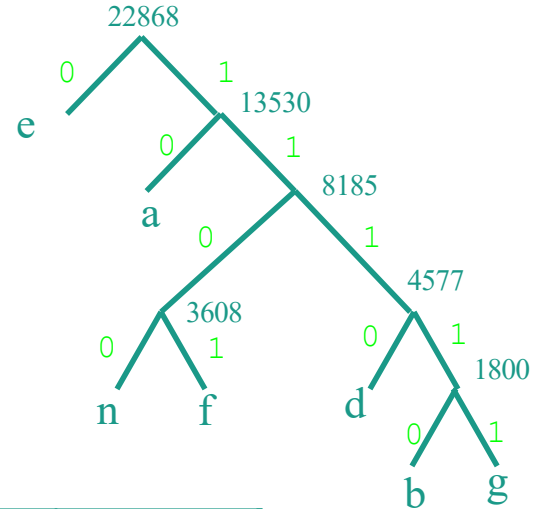
Huffman Encoding

Letter	Occurrence	Percentage
e	9338	0.408344
a	5345	0.233733
n	2420	0.105825
f	2157	0.094324
d	1808	0.079062
b	997	0.043598
g	803	0.035115
Total	22868	100%

Try decoding this:

1111010110010110010

b a n a n a



e	0
a	10
n	1100
f	1101
d	1110
b	11110
g	11111



Huffman Encoding

- ❑ ASCII encoding uses 8 bits per character.
- ❑ Here we use 19 bits for 6 characters ≈ 3.1667 bits per character
 - compression: $1-19/48 \approx 60\%$
- ❑ For a given frequency distribution, there are many possible Huffman codes, but the total compressed length will be the same.
- ❑ <http://www.compressconsult.com/huffman/>

Substitutional Compression



Substitutional Compressors

- ❑ Based on the assumption that a phrase within a message is likely to occur several times
- ❑ Phrases are entered into a **dictionary**
- ❑ Occurrences of a phrase use the dictionary index, rather than the phrase itself
- ❑ Dictionary can be seeded with common phrases
- ❑ Dictionary can be expanded as the message is encoded to include new phrases
- ❑ Dictionary can be implicit

gzip

- ❑ A variation on the LZ77 algorithm
 - Not patented by anyone
- ❑ Dictionary = a window of the most recently processed part of the message
- ❑ If a phrase appears in the window, then output its **(negative) offset and length**
- ❑ If a phrase does not appear, then output its characters
- ❑ Apply Adaptive Huffman Encoding to output!
- ❑ <http://www.gzip.org/algorithm.txt>

gzip

Example: Compress **ABABCBABAB**

-2 -> offset (starting position of the phrase in the window)

2 -> length of the phrase (how many characters)

Characters	Output	Window
A	A	A
B	B	AB
AB	$(-2,2)$	ABAB
C	C	ABABC
BAB	$(-4,3)$	ABABCBAB
AB	$(-2,2)$	ABABCBABAB

gzip

Example: Compress **ABABCBABAB**

Characters	Output	Window
A	A	A
B	B	AB
AB	$(-2,2)$	ABAB
C	C	ABABC
BAB	$(-4,3)$	ABABCBAB
AB	$(-2,2)$	ABABCBABAB

Compressed String: **AB** $(-2,2)$ **C** $(-4,3)$ $(-2,2)$

gzip

Example: Compress **ABABCBABAB**

Characters	Output	Window
A	A	A
B	B	AB
AB	$(-2,2)$	ABAB
C	C	ABABC
BAB	$(-4,3)$	ABABCBAB
AB	$(-2,2)$	ABABCBABAB

Compressed String: **AB** $(-2,2)$ **C** $(-4,3)$ $(-2,2)$

Decompressed String: **AB AB C BAB AB**

gzip

Example: Compress ABABCBABAB

Characters	Output	Window
A	A	A
B	B	AB
AB	$(-2,2)$	ABAB
C	C	ABABC
BAB	$(-4,3)$	ABABCBAB
AB	$(-2,2)$	ABABCBABAB

- Note: blahblahblahblah $\Rightarrow$ blah $(-4,12)$!!



The Best Data Compressor?

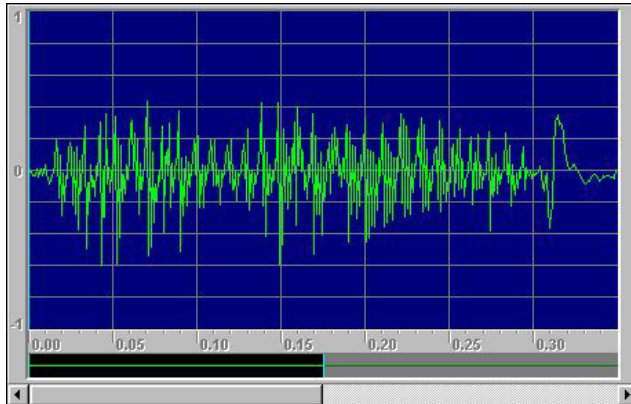
Many factors

- ❑ Speed of method
- ❑ Computing resources required
- ❑ Compression ratio

Compressing Multimedia

Digital Audio

- ❑ Sound is *analogue*
 - potentially infinite number of levels for sound signal



<http://electronics.howstuffworks.com/analog-digital2.htm>

Digital Audio

- ❑ Want to convert into *digital* format
 - finite number of levels
 - some information is lost in conversion

(60min * 60) secs x 44100 samp/sec x 2
bytes/samp x 2 channels

= 635,040,000

About 600 Mbytes (typical CD)

Analogue-to-Digital Converter (ADC)

- ❑ converts analogue signal (sound wave) to produce a digital version
 - Sampling – periodic snapshots
 - Quantisation – number of levels
- ❑ CD stereo– 44.1kHz (44,100 time per second) sampling rate, 16 bits amplitude (2^{16} or 65,536 division of amplitude)
 - = 1400 Kb/second (transfer rate) [storage: 10.1 Mb per minute]
 - AM radio (mono) – 22.05 KHz – 8 bit -22 Kb/sec [1.26 Mb]
 - Telephone (mono) – 11.025KHz – 8 bit – 11 kb/sec [630 Kb]

Digital Audio: MP3

- ❑ Short for **MPEG 1 Layer III**
 - audio component of MPEG's movie compression standard
 - Sampling: 44.1 kHz is common
 - Bit rate: 128 kbits/s (1/11 of CD) at a compression ratio of 11:1 or 320 kbits/s
- ❑ Compression is based on *perceptual audio encoding* or *perceptual noise shaping*
 - Human hearing range (2-5kHz is most sensitive)
 - Psychoacoustic behaviour (loud masks soft)
 - Psychoacoustic model (PAM)
 - Stereo Effects
 - Huffman encoding is applied at the end
 - Not for compression, for encoding

Image: JPEG

- ❑ Joint Photographics Experts Group – committee that wrote the standard
- ❑ Relies on the human eye's inability to discern small changes in colour
- ❑ Compression discards this redundant information
- ❑ Two parts:
 - Baseline lossy algorithm
 - Lossless algorithm is applied to this output

Image: JPEG

- ❑ Steps in algorithm – *changes the picture into a linear set of numbers*
 1. Divides a picture into blocks of 8 x 8 pixel blocks
 2. Group pixels and perform **discrete cosine transform**
 3. Quantisation step – **loses** some info
 4. Encode with **lossless** Huffman technique
 5. Add file headers, decoding table and other decoding information to produce final file.

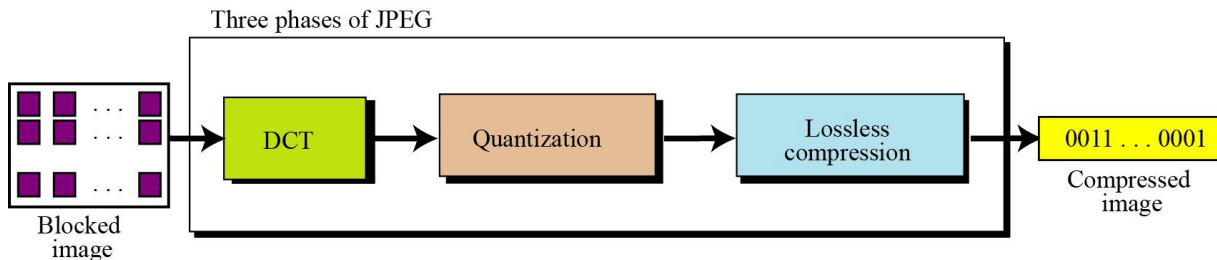


Image: JPEG

Limitations of JPEG compression



Doesn't perform well with high contrast areas

Original

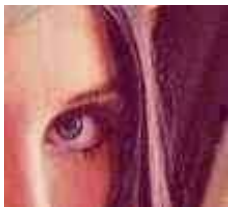
Quality: 0



Multiple applications have an accumulative effect

Original

Max compression



<http://www.cs.cmu.edu/~chuck/lennapg/lenna.shtml>

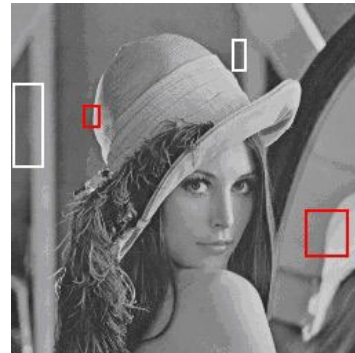
Video: MPEG

- ❑ Moving Picture Experts Group
 - family of standards used for coding audio-visual information in a digital compressed format
 - There are a number of *phases* to the work of the MPEG organization. These are indicated by a number following “MPEG”.
 - MPEG-1** - Video CD, MP3
 - MPEG-2** - Digital Television, set top boxes, DVD
 - MPEG-4** - multimedia for the fixed and mobile web
 - MPEG-7** - description and search of audio and visual content
 - MPEG-DASH** - dynamic streaming

Fractal Compression

- ❑ Fractals are mathematical geometric models which, unlike other geometric models, have infinite detail contained within them.
- ❑ The principle behind fractal based image compression is to find areas in the picture which are similar to other areas in the picture.

- ❑ **Very** slow compression, fast decompression.
- ❑ Extremely high **potential** compression rate.
- ❑ Technology is patented.



Two repeated areas



Summary - Creating our website

- ❑ We have looked at how data is encoded for digital storage and transmission. We have seen how additional information is included at both a per-byte and per-frame level to allow the data to be checked for errors.
- ❑ We have seen how data can be compressed, using statistical methods such as Huffman Coding, and substitutional methods such as LZW and GZIP.
- ❑ Finally, we reviewed some of the modern formats for content compression, for images, video, and audio.
- ❑ Having dealt with the size and speed issues of our website, there is only one final problem that needs to be addressed in order to achieve our goal, that is the matter of making our website secure.
- ❑ We will consider this problem in two ways, securing the data that we send between the client and the server, and securing the actual clients and servers themselves.
- ❑ We will start by looking at how we can secure data that is sent between clients and servers, by the use of encryption – next week.



References

- ❏ Data Communications and Networking (5e)
 - *By Behrouz A. Forouzan*
 - *Chapters 3.1, 4.1, 5.1, 10.1, 10.2, 10.3, 28.1, and 28.2*